

1)An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 19
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE EcommerceDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE EcommerceDB;
Database changed
mysql>
mysql> CREATE TABLE Products (
  ->   Product_ID INT PRIMARY KEY,
  ->   Product_Name VARCHAR(100),
  ->   Category VARCHAR(50),
  ->   Brand VARCHAR(50),
  ->   Price DECIMAL(10,2),
  ->   Hash_Bucket INT
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> ^C
mysql> INSERT INTO Products
  -> (Product_ID, Product_Name, Category, Brand, Price, Hash_Bucket)
  -> VALUES
  -> (1001, 'Laptop', 'Electronics', 'Dell', 55000, MOD(1001,1000)),
  -> (1002, 'Mobile', 'Electronics', 'Samsung', 30000, MOD(1002,1000)),
  -> (1003, 'Headphones', 'Accessories', 'Boat', 2000, MOD(1003,1000)),
  -> (1004, 'Keyboard', 'Accessories', 'HP', 1500, MOD(1004,1000));
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> CREATE INDEX idx_hash_bucket
  -> ON Products(Hash_Bucket);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT *
  -> FROM Products
  -> WHERE Hash_Bucket = MOD(1002,1000)
  -> AND Product_ID = 1002;
+-----+-----+-----+-----+-----+-----+
| Product_ID | Product_Name | Category | Brand | Price | Hash_Bucket |
+-----+-----+-----+-----+-----+-----+
| 1002 | Mobile | Electronics | Samsung | 30000.00 | 2 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_category
  -> ON Products(Category);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_price
  -> ON Products(Price);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT *
  -> FROM Products
  -> WHERE Price BETWEEN 20000 AND 50000;
+-----+-----+-----+-----+-----+-----+
| Product_ID | Product_Name | Category | Brand | Price | Hash_Bucket |
+-----+-----+-----+-----+-----+-----+
| 1002 | Mobile | Electronics | Samsung | 30000.00 | 2 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO Products
  -> VALUES
  -> (1005, 'Smart Watch', 'Wearables', 'Noise', 4000, MOD(1005,1000));
Query OK, 1 row affected (0.01 sec)
```

```
mysql> DELETE FROM Products
-> WHERE Product_ID = 1003;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Products;
```

Product_ID	Product_Name	Category	Brand	Price	Hash_Bucket
1001	Laptop	Electronics	Dell	55000.00	1
1002	Mobile	Electronics	Samsung	30000.00	2
1004	Keyboard	Accessories	HP	1500.00	4
1005	Smart Watch	Wearables	Noise	4000.00	5

```
4 rows in set (0.00 sec)
```

For an e-commerce company storing **10 million product records**, the main requirements are fast product lookup, insertion of new products, and efficient updates/deletions.

1. Recommended Strategy

Use Hash File Organization with:

- **Primary hash key:** Product_ID
- **Hash function:** $h(\text{Product_ID}) = \text{Product_ID} \% N$
- **Collision handling:** Overflow blocks/chaining
- **B+ Tree secondary index:** on attributes frequently searched, such as Category, Brand, or Price.

Eg: Product_ID → Hash Function → Bucket → Product Record

10025 → $h(10025)$ → Bucket 25

20451 → $h(20451)$ → Bucket 51

30567 → $h(30567)$ → Bucket 67

2. Why Hashing?

A product is commonly accessed using its unique **Product_ID**.

Operation	Heap File	Sorted File	Hash File
Search by Product_ID	O(N)	O(log N)	O(1) average
Insert	O(1)	O(N) / maintenance required	O(1) average
Delete	O(N)	O(log N) + maintenance	O(1) average
Sequential scan	Excellent	Excellent	Good
Range search	Poor	Excellent	Poor

For **10 million records**, hash organization avoids scanning millions of records for an exact Product_ID search.

3. Cost Analysis

Assume:

- Number of records = **10,000,000**
- Average record size = **1 KB**
- Total data size $\approx$ **10 GB**
- Block size = **4 KB**
- Records per block $\approx$ 4

Therefore:

Heap File

For an exact search without an index, the system may examine approximately:
blocks on average.

This creates a very high I/O cost.

Hash File

With a well-designed hash function, an exact lookup normally requires approximately:
including possible overflow access.

Thus, the difference is approximately:

for an average exact lookup.

4. Storage and Performance Considerations

The company should allocate enough buckets to maintain a **low load factor**, for example around **0.7–0.8**. This reduces collisions and overflow chains.

For example:

10 million records



Hash Function



Large number of buckets



Low collision rate



Fast Product_ID lookup

A **B+ Tree index** should additionally be maintained for queries such as:

SELECT * FROM Product

WHERE Category = 'Laptop';

or

SELECT * FROM Product

WHERE Price BETWEEN 50000 AND 80000;

Hashing is excellent for **exact-match searches**, while B+ Trees are better for **range searches and ordered retrieval**.

5. Final Recommendation

Hash File Organization with B+ Tree secondary indexes is the most suitable strategy.

Justification:

1. Handles **10 million records** efficiently.
2. Provides approximately **O(1) average** exact-key lookup.
3. Provides fast insertion and deletion.
4. Reduces disk I/O significantly compared with an unindexed heap file.

5. B+ Tree indexes support category, price, and other range-based queries.
6. Overflow buckets can handle hash collisions.

Conclusion

For an e-commerce system where Product_ID lookups are frequent, hashing should be the primary file organization, supplemented by B+ Tree indexes for non-key and range queries. This gives a good balance between storage cost, search performance, and update efficiency.

2) A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE BankingDB;
ERROR 1007 (HY000): Can't create database 'bankingdb'; database exists
mysql> USE BankingDB;
Database changed
mysql> CREATE TABLE Transactions (
->   Transaction_ID INT PRIMARY KEY,
->   Account_No INT NOT NULL,
->   Transaction_Date DATETIME NOT NULL,
->   Transaction_Type VARCHAR(20),
->   Amount DECIMAL(12,2),
->   Balance DECIMAL(12,2)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO Transactions
-> VALUES
-> (1, 10001, '2026-01-05 10:30:00', 'Deposit', 5000, 25000),
-> (2, 10001, '2026-01-10 14:20:00', 'Withdrawal', 2000, 23000),
-> (3, 10002, '2026-01-12 09:15:00', 'Deposit', 10000, 40000),
-> (4, 10001, '2026-01-20 16:00:00', 'Transfer', 3000, 20000),
-> (5, 10002, '2026-01-25 11:45:00', 'Withdrawal', 5000, 35000),
-> (6, 10001, '2026-02-02 13:10:00', 'Deposit', 7000, 27000);
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_account_date
-> ON Transactions(Account_No, Transaction_Date);
Query OK, 0 rows affected (0.05 sec)
```

```

mysql> SELECT *
-> FROM Transactions
-> WHERE Account_No = 10001;
+-----+-----+-----+-----+-----+-----+
| Transaction_ID | Account_No | Transaction_Date | Transaction_Type | Amount | Balance |
+-----+-----+-----+-----+-----+-----+
| 1 | 10001 | 2026-01-05 10:30:00 | Deposit | 5000.00 | 25000.00 |
| 2 | 10001 | 2026-01-10 14:20:00 | Withdrawal | 2000.00 | 23000.00 |
| 4 | 10001 | 2026-01-20 16:00:00 | Transfer | 3000.00 | 20000.00 |
| 6 | 10001 | 2026-02-02 13:10:00 | Deposit | 7000.00 | 27000.00 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT *
-> FROM Transactions
-> WHERE Account_No = 10001
-> AND Transaction_Date
-> BETWEEN '2026-01-01' AND '2026-01-31';
+-----+-----+-----+-----+-----+-----+
| Transaction_ID | Account_No | Transaction_Date | Transaction_Type | Amount | Balance |
+-----+-----+-----+-----+-----+-----+
| 1 | 10001 | 2026-01-05 10:30:00 | Deposit | 5000.00 | 25000.00 |
| 2 | 10001 | 2026-01-10 14:20:00 | Withdrawal | 2000.00 | 23000.00 |
| 4 | 10001 | 2026-01-20 16:00:00 | Transfer | 3000.00 | 20000.00 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Transactions
-> WHERE Account_No = 10001
-> AND Transaction_Date
-> BETWEEN '2026-01-01' AND '2026-01-31';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Transactions | NULL | range | idx_account_date | idx_account_date | 9 | NULL | 3 | 100.00 | Using index condition |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)

```

A banking system may contain millions of transaction records. Common queries are:

- Find all transactions for a particular account number.
- Find transactions for an account within a particular date range.
- Find recent transactions for an account.

For this requirement, a composite B+ Tree index on (Account_No, Transaction_Date) is the most appropriate choice.

2. Table Structure

Assume the transaction table contains:

Attribute	Description
Transaction_ID	Unique transaction identifier
Account_No	Bank account number
Transaction_Date	Date/time of transaction
Transaction_Type	Deposit/Withdrawal/Transfer
Amount	Transaction amount
Balance	Account balance after transaction

3. Recommended Index

Create a composite index:

(Account_No, Transaction_Date)

Why this order?

Account_No



Transaction_Date

The database first locates the required account and then efficiently searches the dates belonging to that account.

For example:

```
SELECT *
```

```
FROM Transactions
```

```
WHERE Account_No = 10001
```

```
AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';
```

Without an index, the database may need to scan millions of records.

With the composite index, it can directly locate the relevant portion of the index.

4. Why B+ Tree?

A B+ Tree index is suitable because banking queries involve both:

- Exact matching: Account_No = 10001
- Range searching: Transaction_Date BETWEEN ... AND ...

Hash indexes are good for exact matches but are not suitable for efficient date-range queries.

Therefore:

B+ Tree + composite index (Account_No, Transaction_Date) is the recommended strategy.

3) A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

ANS

```
MySQL 8.0 Command Line Cli  X  +  v

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE HealthcareDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE HealthcareDB;
Database changed
mysql> CREATE TABLE Patients (
  -> PatientID INT PRIMARY KEY,
  -> PatientName VARCHAR(100),
  -> DoctorName VARCHAR(100),
  -> Department VARCHAR(100),
  -> Diagnosis VARCHAR(200)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO Patients
  -> VALUES
  -> (101, 'Ravi', 'Dr. Kumar', 'Cardiology', 'Blood Pressure'),
  -> (102, 'Priya', 'Dr. Sharma', 'Neurology', 'Migraine'),
  -> (103, 'Arun', 'Dr. Kumar', 'Cardiology', 'Diabetes'),
  -> (104, 'Meena', 'Dr. Rao', 'Orthopedics', 'Fracture'),
  -> (105, 'Anu', 'Dr. Kumar', 'Cardiology', 'Blood Pressure');
Query OK, 5 rows affected (0.02 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_patient_id
  -> ON Patients(PatientID);
Query OK, 0 rows affected (0.11 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM Patients
  -> WHERE PatientID = 103;
+-----+-----+-----+-----+-----+
| PatientID | PatientName | DoctorName | Department | Diagnosis |
+-----+-----+-----+-----+-----+
| 103 | Arun | Dr. Kumar | Cardiology | Diabetes |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_doctor_name
  -> ON Patients(DoctorName);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> SELECT *
  -> FROM Patients
  -> WHERE DoctorName = 'Dr. Kumar';
+-----+-----+-----+-----+-----+
| PatientID | PatientName | DoctorName | Department | Diagnosis |
+-----+-----+-----+-----+-----+
| 101 | Ravi | Dr. Kumar | Cardiology | Blood Pressure |
| 103 | Arun | Dr. Kumar | Cardiology | Diabetes |
| 105 | Anu | Dr. Kumar | Cardiology | Blood Pressure |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> CREATE INDEX idx_doctor_patient
  -> ON Patients(DoctorName, PatientID);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM Patients
  -> WHERE DoctorName = 'Dr. Kumar'
  -> ORDER BY PatientID;
+-----+-----+-----+-----+-----+
| PatientID | PatientName | DoctorName | Department | Diagnosis |
+-----+-----+-----+-----+-----+
| 101 | Ravi | Dr. Kumar | Cardiology | Blood Pressure |
| 103 | Arun | Dr. Kumar | Cardiology | Diabetes |
| 105 | Anu | Dr. Kumar | Cardiology | Blood Pressure |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SHOW INDEX FROM Patients;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| patients | 0 | PRIMARY | 1 | PatientID | A | 5 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| patients | 1 | idx_patient_id | 1 | PatientID | A | 5 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| patients | 1 | idx_doctor_name | 1 | DoctorName | A | 3 | NULL | NULL | YES | BTREE | | | YES | NULL |
| patients | 1 | idx_doctor_patient | 1 | DoctorName | A | 3 | NULL | NULL | YES | BTREE | | | YES | NULL |
| patients | 1 | idx_doctor_patient | 2 | PatientID | A | 5 | NULL | NULL | NULL | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM Patients
  -> WHERE PatientID = 103;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Patients | NULL | const | PRIMARY, idx_patient_id | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM Patients
  -> WHERE DoctorName = 'Dr. Kumar';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Patients | NULL | ref | idx_doctor_name, idx_doctor_patient | idx_doctor_name | 403 | const | 3 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

A healthcare system may store millions of patient records. The records need to be accessed in two important ways:

1. **PatientID** → to quickly find one specific patient's record.

2. **Doctor Name** → to find all patients associated with a particular doctor.

Therefore, a **multi-level indexing strategy** is appropriate.

Recommended Index Structure

Use:

- **Primary B+ Tree index on PatientID**
- **Secondary B+ Tree index on Doctor_Name**
- Optionally, use a **composite index (Doctor_Name, PatientID)** if the system frequently retrieves patients of a doctor and then sorts/searches by PatientID.

Why B+ Tree?

B+ Trees are suitable because they support:

- Fast exact-match searches
- Range searches
- Ordered traversal
- Large datasets
- Efficient disk-based access

2. Example Patient Table

PatientID	PatientName	Doctor_Name	Department	Diagnosis
101	Ravi	Dr. Kumar	Cardiology	BP
102	Priya	Dr. Sharma	Neurology	Migraine
103	Arun	Dr. Kumar	Cardiology	Diabetes
104	Meena	Dr. Rao	Orthopedics	Fracture
105	Anu	Dr. Kumar	Cardiology	BP

4) A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE LogisticsDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE LogisticsDB;
Database changed
mysql> CREATE TABLE Shipments (
  ->   ShipmentID INT PRIMARY KEY,
  ->   CustomerName VARCHAR(100),
  ->   Source VARCHAR(100),
  ->   Destination VARCHAR(100),
  ->   ShipmentDate DATE,
  ->   Status VARCHAR(30),
  ->   Weight DECIMAL(8,2)
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Shipments
  -> VALUES
  -> (1001, 'Ravi', 'Chennai', 'Bangalore',
  ->  '2026-08-01', 'In Transit', 15.50),
  ->
  -> (1002, 'Priya', 'Hyderabad', 'Chennai',
  ->  '2026-08-03', 'Delivered', 8.20),
  ->
  -> (1003, 'Arun', 'Chennai', 'Mumbai',
  ->  '2026-08-05', 'In Transit', 20.00),
  ->
  -> (1004, 'Meena', 'Delhi', 'Pune',
  ->  '2026-08-07', 'Pending', 12.75),
  ->
  -> (1005, 'Anu', 'Chennai', 'Delhi',
  ->  '2026-08-10', 'In Transit', 18.40);
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> CREATE INDEX idx_shipment_date
  -> ON Shipments(ShipmentDate);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_destination
  -> ON Shipments(Destination);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_date_status
  -> ON Shipments(ShipmentDate, Status);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT *
  -> FROM Shipments
  -> WHERE ShipmentID = 1003;
+-----+-----+-----+-----+-----+-----+
| ShipmentID | CustomerName | Source | Destination | ShipmentDate | Status | Weight |
+-----+-----+-----+-----+-----+-----+
| 1003 | Arun | Chennai | Mumbai | 2026-08-05 | In Transit | 20.00 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
  -> FROM Shipments
  -> WHERE ShipmentDate
  -> BETWEEN '2026-08-01' AND '2026-08-07';
+-----+-----+-----+-----+-----+-----+
| ShipmentID | CustomerName | Source | Destination | ShipmentDate | Status | Weight |
+-----+-----+-----+-----+-----+-----+
| 1001 | Ravi | Chennai | Bangalore | 2026-08-01 | In Transit | 15.50 |
| 1002 | Priya | Hyderabad | Chennai | 2026-08-03 | Delivered | 8.20 |
| 1003 | Arun | Chennai | Mumbai | 2026-08-05 | In Transit | 20.00 |
| 1004 | Meena | Delhi | Pune | 2026-08-07 | Pending | 12.75 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql> INSERT INTO Shipments
  -> VALUES
  -> (1006, 'Kiran', 'Bangalore', 'Kolkata',
  ->  '2026-08-15', 'Pending', 10.50);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> DELETE FROM Shipments
-> WHERE ShipmentID = 1002;
Query OK, 1 row affected (0.01 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Shipments
-> WHERE ShipmentDate
-> BETWEEN '2026-08-01' AND '2026-08-07';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Shipments	NULL	range	idx_shipment_date,idx_date_status	idx_shipment_date	4	NULL	3	100.00	Using index condition

1. Problem

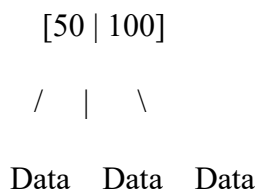
A logistics company continuously:

- Inserts new shipment records.
- Deletes completed/cancelled shipments.
- Searches shipments using ShipmentID.
- Retrieves shipments over a range of dates or locations.
- Performs sequential/range scans for reports.

Because the data changes frequently, the index must support efficient insertion, deletion, and searching.

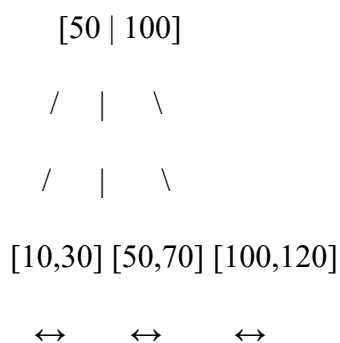
2. B-Tree vs B+ Tree

B-Tree: A B-Tree stores keys and actual record pointers in both internal and leaf nodes.



B+ Tree: A B+ Tree stores actual record pointers only in leaf nodes. Internal nodes contain only search keys.

The leaf nodes are linked.



3. Comparison

Feature	B-Tree	B+ Tree
Search	Fast	Fast
Insert	Efficient	Efficient
Delete	Efficient	Efficient
Range queries	Good	Excellent
Sequential access	Good	Excellent
Leaf nodes linked	No	Yes
Internal nodes contain records	Yes	No
Fan-out	Lower	Higher
Disk I/O	Good	Usually lower
Database systems	Less common	Very common

4. Which is Better?

B+ Tree is more suitable.

For a logistics company, shipment records are frequently inserted and deleted. At the same time, the company may need queries such as:

Find shipment 10025 or Find all shipments between January 1 and January 31.

B+ Trees are particularly effective for these workloads because:

1. High fan-out reduces tree height.
2. Linked leaf nodes make range queries very efficient.
3. Insertions and deletions are handled through node splitting and merging/redistribution.
4. All records are located at the leaf level, making traversal predictable.
5. They are well suited for disk-based database indexes.

5) Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 23
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE HRDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE HRDB;
Database changed
mysql> CREATE TABLE Employees (
  -> Employee_ID INT PRIMARY KEY,
  -> Employee_Name VARCHAR(100),
  -> Department VARCHAR(50),
  -> Salary DECIMAL(10,2),
  -> Hash_Bucket INT
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO Employees
  -> VALUES
  -> (1001, 'Ravi', 'IT', 50000, MOD(1001,1000)),
  -> (1002, 'Priya', 'HR', 45000, MOD(1002,1000)),
  -> (1250, 'Arun', 'Finance', 55000, MOD(1250,1000)),
  -> (2250, 'Meena', 'IT', 60000, MOD(2250,1000)),
  -> (3501, 'Anu', 'Marketing', 48000, MOD(3501,1000));
Query OK, 5 rows affected (0.05 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_hash_bucket
  -> ON Employees(Hash_Bucket);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT *
  -> FROM Employees
  -> WHERE Hash_Bucket = MOD(1250,1000)
  -> AND Employee_ID = 1250;
+-----+-----+-----+-----+-----+
| Employee_ID | Employee_Name | Department | Salary | Hash_Bucket |
+-----+-----+-----+-----+-----+
| 1250 | Arun | Finance | 55000.00 | 250 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO Employees
  -> VALUES
  -> (4500, 'Kiran', 'IT', 52000, MOD(4500,1000));
Query OK, 1 row affected (0.01 sec)

mysql> DELETE FROM Employees
  -> WHERE Employee_ID = 1250;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *
  -> FROM Employees
  -> WHERE Hash_Bucket = MOD(3501,1000)
  -> AND Employee_ID = 3501;
+-----+-----+-----+-----+-----+
| Employee_ID | Employee_Name | Department | Salary | Hash_Bucket |
+-----+-----+-----+-----+-----+
| 3501 | Anu | Marketing | 48000.00 | 501 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

1. Problem

An HR system stores 5,000 employee records. Employees are frequently searched using their unique Employee_ID.

Typical operations are:

- Insert a new employee
- Search employee by ID

- Delete an employee
- Update employee details

2. Proposed Hashing Scheme

Use:

Hash function:

This creates 1000 buckets.

Each bucket can store approximately:

employees on average.

Employee_ID



Hash Function

$h(ID) = ID \text{ MOD } 1000$



Bucket



Employee Record

For example:

Employee ID = 1250

$1250 \text{ MOD } 1000 = 250$

Therefore:

Employee 1250 → Bucket 250

3. Static Hashing vs Extendible Hashing

Feature	Static Hashing	Extendible Hashing
Number of buckets	Fixed	Dynamic
Growth handling	Overflow buckets required	Buckets split automatically
Implementation	Simple	More complex
Performance	Good if size is stable	Good for changing data
Overflow problem	Possible	Reduced
Memory overhead	Low	Higher
Best for	Fixed-size datasets	Frequently growing datasets

For 5,000 employees

If the HR system is expected to remain approximately 5,000 employees, static hashing is sufficient and simpler.

However, if the company expects rapid growth—for example:

5000 → 10000 → 20000 employees

then extendible hashing is preferable because the hash structure can grow dynamically.

Recommendation

For this scenario, choose static hashing initially, because:

- Employee count is specified as 5,000.
- The structure is simple.
- Exact Employee_ID searches are very fast.
- There is less implementation overhead.

6) An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

ANS

```
MySQL 8.0 Command Line Cli  X  +  v

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 24
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE AirlineDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE AirlineDB;
Database changed
mysql> CREATE TABLE Flights (
  ->   Flight_ID INT PRIMARY KEY,
  ->   Flight_Number VARCHAR(20) NOT NULL,
  ->   Source VARCHAR(50),
  ->   Destination VARCHAR(50),
  ->   Flight_Date DATE NOT NULL,
  ->   Departure_Time TIME,
  ->   Arrival_Time TIME,
  ->   Seats_Available INT
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO Flights
  -> VALUES
  -> (1, 'AI101', 'Chennai', 'Delhi',
  ->  '2026-08-01', '08:00:00', '11:00:00', 120),
  ->
  -> (2, 'AI202', 'Mumbai', 'Delhi',
  ->  '2026-08-05', '09:30:00', '11:30:00', 80),
  ->
  -> (3, '6E305', 'Chennai', 'Mumbai',
  ->  '2026-08-10', '14:00:00', '16:00:00', 95),
  ->
  -> (4, 'AI404', 'Delhi', 'Bangalore',
  ->  '2026-08-15', '18:00:00', '21:00:00', 60),
  ->
  -> (5, '6E505', 'Hyderabad', 'Chennai',
  ->  '2026-08-20', '07:00:00', '08:30:00', 100),
  ->
  -> (6, 'AI606', 'Mumbai', 'Kolkata',
  ->  '2026-08-25', '16:00:00', '19:00:00', 75);
Query OK, 6 rows affected (0.05 sec)
```

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> CREATE INDEX idx_flight_number
  -> ON Flights(Flight_Number);
Query OK, 0 rows affected (0.11 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM Flights
  -> WHERE Flight_Number = 'AI202';

+----+-----+-----+-----+-----+-----+-----+-----+
| Flight_ID | Flight_Number | Source | Destination | Flight_Date | Departure_Time | Arrival_Time | Seats_Available |
+----+-----+-----+-----+-----+-----+-----+-----+
| 2 | AI202 | Mumbai | Delhi | 2026-08-05 | 09:30:00 | 11:30:00 | 80 |
+----+-----+-----+-----+-----+-----+-----+-----+

1 row in set (0.00 sec)

mysql> CREATE UNIQUE INDEX idx_flight_number
  -> ON Flights(Flight_Number);
ERROR 1061 (20000): Duplicate key name 'idx_flight_number'

mysql> SELECT *
  -> FROM Flights
  -> WHERE Flight_Date
  -> BETWEEN '2026-08-01' AND '2026-08-15';

+----+-----+-----+-----+-----+-----+-----+-----+
| Flight_ID | Flight_Number | Source | Destination | Flight_Date | Departure_Time | Arrival_Time | Seats_Available |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | AI101 | Chennai | Delhi | 2026-08-01 | 08:00:00 | 11:00:00 | 120 |
| 2 | AI202 | Mumbai | Delhi | 2026-08-05 | 09:30:00 | 11:30:00 | 80 |
| 3 | 6E305 | Chennai | Mumbai | 2026-08-10 | 14:00:00 | 16:00:00 | 95 |
| 4 | AI404 | Delhi | Bangalore | 2026-08-15 | 18:00:00 | 21:00:00 | 60 |
+----+-----+-----+-----+-----+-----+-----+-----+

4 rows in set (0.00 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM Flights
  -> WHERE Flight_Number = 'AI202';

+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Flights | NULL | ref | idx_flight_number | idx_flight_number | 82 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM Flights
  -> WHERE Flight_Date
  -> BETWEEN '2026-08-01' AND '2026-08-15';

+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Flights | NULL | ALL | NULL | NULL | NULL | NULL | 6 | 16.67 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

1 row in set, 1 warning (0.00 sec)

mysql> SELECT *
  -> FROM Flights
  -> WHERE Flight_Number = 'AI202'
  -> AND Flight_Date = '2026-08-05';

+----+-----+-----+-----+-----+-----+-----+-----+
| Flight_ID | Flight_Number | Source | Destination | Flight_Date | Departure_Time | Arrival_Time | Seats_Available |
+----+-----+-----+-----+-----+-----+-----+-----+
| 2 | AI202 | Mumbai | Delhi | 2026-08-05 | 09:30:00 | 11:30:00 | 80 |
+----+-----+-----+-----+-----+-----+-----+-----+

1 row in set (0.01 sec)
```

1. Problem

An airline reservation system needs two important types of queries:

1. **Point query:** Find a specific flight using Flight_Number.
2. **Range query:** Find flights operating between two dates using Flight_Date.

Example:

-- Point query

SELECT *

FROM Flights

WHERE Flight_Number = 'AI202';

-- Range query

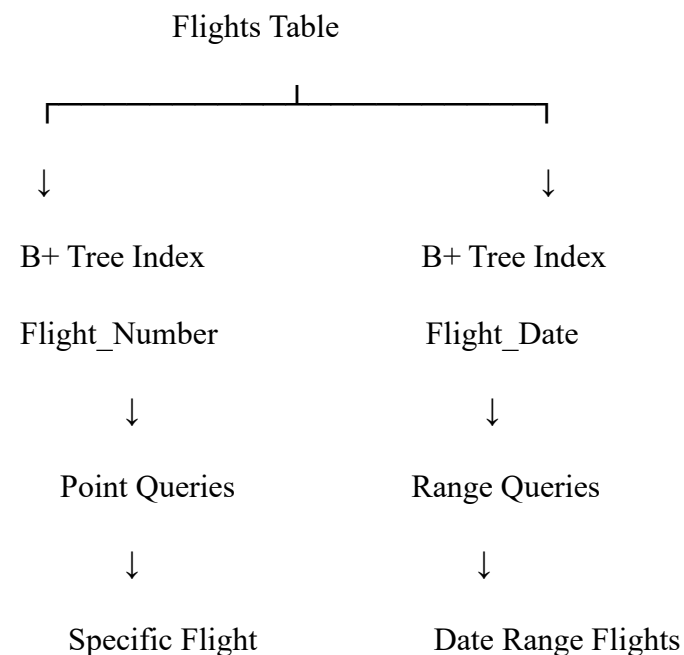
SELECT *

FROM Flights

WHERE Flight_Date BETWEEN '2026-08-01' AND '2026-08-31';

Since these queries have different access patterns, a combined indexing strategy is appropriate.

2. Proposed Index Structure : Use two B+ Tree indexes:



Index 1: Flight Number

INDEX(Flight_Number)

Used for exact/point searches.

Index 2: Flight Date

INDEX(Flight_Date)

Used for range searches.

3. Why B+ Tree?

B+ Tree is suitable for both requirements:

Query	Suitable Index	Reason
Flight_Number = 'AI202'	B+ Tree on Flight_Number	Fast exact lookup
Flight_Date BETWEEN ...	B+ Tree on Flight_Date	Efficient range scan
Date sorting	B+ Tree on Flight_Date	Maintains ordered keys
Sequential date retrieval	B+ Tree	Linked leaf nodes

A hash index would be good for the flight-number point query but not suitable for date-range queries. Therefore, B+ Trees provide a better combined solution.

7) For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

ANS

```
MySQL 8.0 Command Line Cli X + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 25
Server version: 8.0.46 MySQL Community Server - GPL
Copyright (c) 2000, 2020, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE UniversityDB;
ERROR 1007 (HY000): Can't create database 'UniversityDB'; database exists
mysql> USE UniversityDB;
Database changed
mysql> CREATE TABLE Students (
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(100),
  -> Department VARCHAR(50),
  -> CGPA DECIMAL(3,2),
  -> Year INT
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Students
  -> VALUES
  -> (101, 'Aishwarya', 'CSE', 8.75, 2),
  -> (102, 'Rahul', 'ECE', 7.80, 3),
  -> (103, 'Priya', 'CSE', 9.10, 2),
  -> (104, 'Arun', 'IT', 8.20, 3),
  -> (105, 'Meena', 'CSE', 8.40, 4),
  -> (106, 'Kiran', 'EEE', 7.50, 2),
  -> (107, 'Anu', 'IT', 9.00, 4),
  -> (108, 'Ravi', 'CSE', 7.90, 3);
Query OK, 8 rows affected (0.05 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_department
  -> ON Students(Department);
Query OK, 0 rows affected (0.15 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE';
+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | CGPA | Year |
+-----+-----+-----+-----+-----+
| 101 | Aishwarya | CSE | 8.75 | 2 |
| 103 | Priya | CSE | 9.10 | 2 |
| 105 | Meena | CSE | 8.40 | 4 |
| 108 | Ravi | CSE | 7.90 | 3 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli X + v

mysql> CREATE INDEX idx_cgpa
  -> ON Students(CGPA);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM Students
  -> WHERE CGPA BETWEEN 8.00 AND 9.00;
+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | CGPA | Year |
+-----+-----+-----+-----+-----+
| 104 | Arun | IT | 8.20 | 3 |
| 105 | Meena | CSE | 8.40 | 4 |
| 101 | Aishwarya | CSE | 8.75 | 2 |
| 107 | Anu | IT | 9.00 | 4 |
+-----+-----+-----+-----+-----+
4 rows in set (0.04 sec)

mysql> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE'
  -> AND CGPA BETWEEN 8.00 AND 9.00;
+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | CGPA | Year |
+-----+-----+-----+-----+-----+
| 101 | Aishwarya | CSE | 8.75 | 2 |
| 105 | Meena | CSE | 8.40 | 4 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> CREATE INDEX idx_department_cgpa
  -> ON Students(Department, CGPA);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE'
  -> AND CGPA BETWEEN 8.00 AND 9.00;
+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | CGPA | Year |
+-----+-----+-----+-----+-----+
| 101 | Aishwarya | CSE | 8.75 | 2 |
| 105 | Meena | CSE | 8.40 | 4 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SHOW INDEX FROM Students;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| students | 0 | PRIMARY | 1 | Student_ID | A | 8 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| students | 1 | idx_department | 1 | Department | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| students | 1 | idx_cgpa | 1 | CGPA | A | 8 | NULL | NULL | YES | BTREE | | | YES | NULL |
| students | 1 | idx_department_cgpa | 1 | Department | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| students | 1 | idx_department_cgpa | 2 | CGPA | A | 8 | NULL | NULL | YES | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

```

mysql> EXPLAIN
-> SELECT *
-> FROM Students
-> WHERE Department = 'CSE';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Students | NULL | ref | idx_department,idx_department_cgpa | idx_department | 203 | const | 4 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Students
-> WHERE CGPA BETWEEN 8.00 AND 9.00;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Students | NULL | range | idx_cgpa | idx_cgpa | 3 | NULL | 4 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Students
-> WHERE Department = 'CSE'
-> AND CGPA BETWEEN 8.00 AND 9.00;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Students | NULL | ref | idx_department,idx_cgpa,idx_department_cgpa | idx_department | 203 | const | 4 | 50.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

1. Problem

A university database may contain thousands or millions of student records. The system needs to efficiently answer queries such as:

1. Find all students from a particular department.
2. Find students whose CGPA falls within a given range.

For example:

SELECT *

FROM Students

WHERE Department = 'CSE';

and:

SELECT *

FROM Students

WHERE CGPA BETWEEN 8.0 AND 9.0;

Therefore, the database should use an appropriate file organization along with secondary indexes.

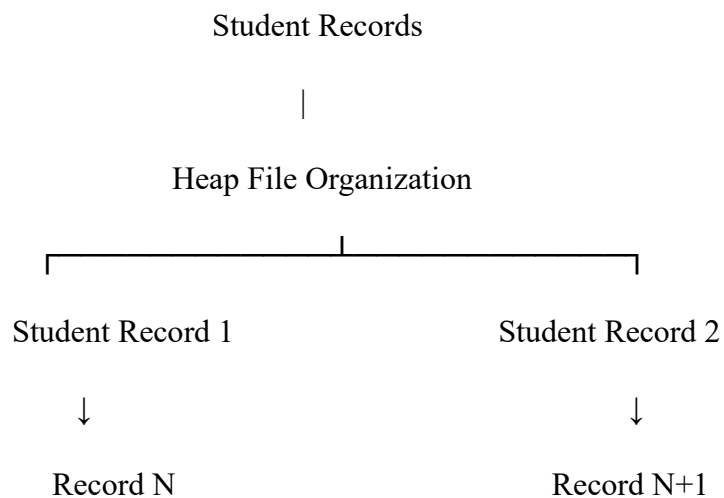
2. Recommended File Organization

Heap File Organization

Use heap (unordered) file organization for the main student table.

In heap organization:

- Records are stored without requiring a particular ordering.
- New student records can be inserted quickly.
- The DBMS manages the physical storage.
- Secondary indexes provide fast access for frequently searched attributes.

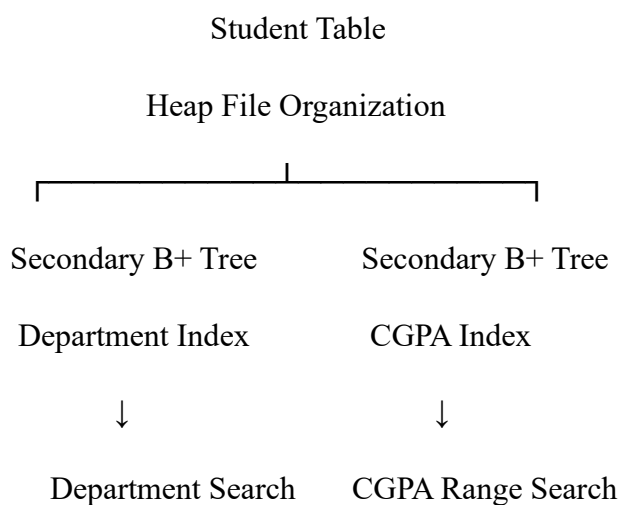


Heap organization is suitable because students can be continuously added, updated, or removed.

3. Secondary Indexing Strategy

Create **secondary B+ Tree indexes** on: Department & CGPA

The structure becomes:



Why B+ Tree?

B+ Tree indexes are suitable because:

- Department supports equality searches.
- CGPA supports range searches.
- They provide ordered traversal.
- They reduce disk I/O compared with scanning the entire table.

8) Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 27
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE FileOrgDB;
Query OK, 1 row affected (0.10 sec)

mysql> USE FileOrgDB;
Database changed
mysql> CREATE TABLE HeapFile (
->   ID INT PRIMARY KEY,
->   Data VARCHAR(42)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.21 sec)

mysql> INSERT INTO HeapFile (ID, Data)
-> WITH RECURSIVE numbers AS (
->   SELECT 1 AS n
->   UNION ALL
->   SELECT n + 1
->   FROM numbers
->   WHERE n < 100000
-> )
-> SELECT n, RPAD(CONCAT('Record', n), 42, 'X')
-> FROM numbers;
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increasing @@cte_max_recursion_depth to a larger value.
```

```
MySQL 8.0 Command Line Cli x + v
mysql> SELECT COUNT(*) AS Total_Records
-> FROM HeapFile;
+-----+
| Total_Records |
+-----+
|              0 |
+-----+
1 row in set (0.01 sec)

mysql> CREATE TABLE SortedFile (
->   ID INT PRIMARY KEY,
->   Data VARCHAR(42)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO SortedFile
-> SELECT ID, Data
-> FROM HeapFile
-> ORDER BY ID;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM SortedFile
-> ORDER BY ID
-> LIMIT 10;
Empty set (0.02 sec)

mysql> CREATE TABLE HashedFile (
->   ID INT PRIMARY KEY,
->   Data VARCHAR(42),
->   Hash_Bucket INT
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO HashedFile (ID, Data, Hash_Bucket)
-> SELECT ID, Data, MOD(ID, 1000)
-> FROM HeapFile;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli x + v
mysql> SELECT COUNT(*) AS Total_Records
-> FROM HeapFile;
+-----+
| Total_Records |
+-----+
| 0 |
+-----+
1 row in set (0.01 sec)

mysql> CREATE TABLE SortedFile (
-> ID INT PRIMARY KEY,
-> Data VARCHAR(42)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO SortedFile
-> SELECT ID, Data
-> FROM HeapFile
-> ORDER BY ID;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM SortedFile
-> ORDER BY ID
-> LIMIT 10;
Empty set (0.02 sec)

mysql> CREATE TABLE HashedFile (
-> ID INT PRIMARY KEY,
-> Data VARCHAR(42),
-> Hash_Bucket INT
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO HashedFile (ID, Data, Hash_Bucket)
-> SELECT ID, Data, MOD(ID, 1000)
-> FROM HeapFile;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT Hash_Bucket, COUNT(*) AS Records
-> FROM HashedFile
-> GROUP BY Hash_Bucket
-> ORDER BY Hash_Bucket
-> LIMIT 10;
Empty set (0.04 sec)
```

```
MySQL 8.0 Command Line Cli x + v
mysql> CREATE INDEX idx_hash_bucket
-> ON HashedFile(Hash_Bucket);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM HashedFile
-> WHERE Hash_Bucket = MOD(50025, 1000)
-> AND ID = 50025;
Empty set (0.01 sec)

mysql> SELECT
-> 4096 AS Block_Size,
-> 50 AS Record_Size,
-> FLOOR(4096 / 50) AS Records_Per_Block,
-> CEIL(100000 / FLOOR(4096 / 50)) AS Total_Blocks,
-> 100000 * 50 AS Data_Size,
-> CEIL(100000 / FLOOR(4096 / 50)) * 4096 AS Allocated_Space;
+-----+-----+-----+-----+-----+
| Block_Size | Record_Size | Records_Per_Block | Total_Blocks | Data_Size | Allocated_Space |
+-----+-----+-----+-----+-----+
| 4096 | 50 | 81 | 1235 | 5000000 | 5048560 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT
-> ROUND(
-> (100000 * 50) /
-> (CEIL(100000 / FLOOR(4096 / 50)) * 4096) * 100,
-> 2
-> ) AS Block_Utilization_Percent;
+-----+
| Block_Utilization_Percent |
+-----+
| 98.84 |
+-----+
1 row in set (0.04 sec)

mysql> CREATE TABLE FileOrganizationAnalysis (
-> File_Organization VARCHAR(20),
-> Number_of_Records INT,
-> Record_Size INT,
-> Block_Size INT,
-> Records_Per_Block INT,
-> Number_of_Blocks INT,
-> Utilization_Percent DECIMAL(5,2)
-> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO FileOrganizationAnalysis
-> VALUES
-> ('Heap', 100000, 50, 4096, 81, 1235, 98.84),
-> ('Sorted', 100000, 50, 4096, 81, 1235, 98.84),
-> ('Hashed', 100000, 50, 4096, 81, 1235, 98.84);
Query OK, 3 rows affected (0.07 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM FileOrganizationAnalysis;
+-----+-----+-----+-----+-----+-----+
| File_Organization | Number_of_Records | Record_Size | Block_Size | Records_Per_Block | Number_of_Blocks | Utilization_Percent |
+-----+-----+-----+-----+-----+-----+
| Heap | 100000 | 50 | 4096 | 81 | 1235 | 98.84 |
| Sorted | 100000 | 50 | 4096 | 81 | 1235 | 98.84 |
| Hashed | 100000 | 50 | 4096 | 81 | 1235 | 98.84 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

File organization defines how records are physically stored in database blocks.

The three common organizations are:

1. Heap File Organization
2. Sorted File Organization
3. Hashed File Organization

For this problem:

- Number of records = 100,000
- Record size = 50 bytes
- Block size = 4 KB = 4096 bytes

The objective is to determine the number of records that fit in each block, total blocks required, and block utilization.

2. Records per Block

The number of records that can fit into one block is:

Therefore, 81 records can be stored in one block.

Space occupied by 81 records:

Unused space:

Block utilization:

3. Total Number of Blocks

For 100,000 records:

So, 1235 blocks are required.

There will be:

- 1234 completely filled blocks
- 1 partially filled block

The final block contains:

records.

4. Heap File Organization

In a Heap File, records are stored in no particular order. New records are generally placed wherever suitable free space is available.

Example

Block 1 → Record 1 – 81

Block 2 → Record 82 – 162

Block 3 → Record 163 – 243

...

Block 1235 → Remaining records

Characteristics

- Simple implementation
- Fast insertion
- Good for sequential scanning
- Searching for a particular record can be slow without an index
- Records are not sorted

Block Utilization

For the given fixed-size records:

5. Sorted File Organization

In a Sorted File, records are stored according to a search key.

For example:

Student_ID

or:

Employee_ID

Records are maintained in sorted order.

Example

Block 1 → 1, 2, 3, ..., 81

Block 2 → 82, 83, ..., 162

Advantages

- Efficient searching
- Excellent for range queries
- Supports ordered sequential processing
- Binary search can be used

Disadvantages

- Insertion is expensive because records may need to be shifted
- Deletion may require reorganization
- Maintaining sorted order has additional cost

Block Utilization

Since the records have the same fixed size:

6. Hashed File Organization

In a Hashed File, a hash function determines the block or bucket where a record is stored.

For example:

where B is the number of buckets.

Example:

Employee ID = 1250

$1250 \text{ MOD } 1000 = 250$

Therefore:

Employee 1250 $\rightarrow$ Bucket 250

Advantages

- Very fast exact-match search
- Fast insertion
- Fast deletion

- Average search complexity is approximately $O(1)$

Disadvantages

- Collisions may occur
- Overflow blocks may be required
- Poor for range queries
- Actual utilization depends on the hash function and load factor

Under the ideal assumption of uniform distribution and no overflow, the basic block utilization is the same as the other organizations:

7. Comparison

Feature	Heap	Sorted	Hashed
Record order	Unordered	Sorted	Hash-based
Insertion	Fast	Slow	Fast
Exact search	Slow without index	Good	Very fast
Range search	Slow	Very fast	Poor
Sequential scan	Good	Good	Good
Collision	No	No	Possible
Block utilization	~98.84%	~98.84%	~98.84%*

* Ideal case without overflow blocks.

8. Cost Calculation

Actual data size:

Allocated space:

Unused space:

Overall utilization:

9. Important Observation

The block utilization is essentially the same for Heap, Sorted, and ideal Hashed organization because all three store the same 100,000 fixed-size records.

The main difference is performance, not basic storage utilization:

- Heap → best for simple insertion and sequential access.
- Sorted → best for ordered and range queries.
- Hashed → best for exact-match searches.

9) A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

ANS

```
MySQL 8.0 Command Line CLI
Enter password: *****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SocialMediaDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE SocialMediaDB;
Database changed
mysql> CREATE TABLE Posts (
  ->   post_id BIGINT PRIMARY KEY,
  ->   user_id BIGINT NOT NULL,
  ->   post_text TEXT,
  ->   timestamp DATETIME NOT NULL,
  ->   hashtag VARCHAR(100)
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Posts
  -> VALUES
  -> (1, 101, 'Learning DBMS', '2026-08-01 10:00:00', '#DBMS'),
  -> (2, 101, 'Learning AI', '2026-08-05 12:00:00', '#AI'),
  -> (3, 102, 'Cloud computing', '2026-08-10 15:00:00', '#Cloud'),
  -> (4, 101, 'Machine Learning', '2026-08-15 09:30:00', '#ML'),
  -> (5, 103, 'Database indexing', '2026-08-20 18:00:00', '#DBMS');
Query OK, 5 rows affected (0.02 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_user_timestamp
  -> ON Posts(user_id, timestamp);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> SELECT *
-> FROM Posts
-> WHERE user_id = 101
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-31';
+-----+-----+-----+-----+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+-----+-----+-----+-----+
| 1 | 101 | Learning DBMS | 2026-08-01 10:00:00 | #DBMS |
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
| 4 | 101 | Machine Learning | 2026-08-15 09:30:00 | #ML |
+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> CREATE INDEX idx_hashtag_timestamp
-> ON Posts(hashtag, timestamp);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM Posts
-> WHERE hashtag = '#AI'
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-31';
+-----+-----+-----+-----+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+-----+-----+-----+-----+
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
+-----+-----+-----+-----+-----+
1 row in set (0.04 sec)

mysql> SELECT *
-> FROM Posts
-> WHERE user_id = 101;
+-----+-----+-----+-----+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+-----+-----+-----+-----+
| 1 | 101 | Learning DBMS | 2026-08-01 10:00:00 | #DBMS |
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
| 4 | 101 | Machine Learning | 2026-08-15 09:30:00 | #ML |
+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> SELECT *
-> FROM Posts
-> WHERE user_id = 101
-> ORDER BY timestamp DESC
-> LIMIT 20;
+-----+-----+-----+-----+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+-----+-----+-----+-----+
| 4 | 101 | Machine Learning | 2026-08-15 09:30:00 | #ML |
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
| 1 | 101 | Learning DBMS | 2026-08-01 10:00:00 | #DBMS |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT *
-> FROM Posts
-> WHERE hashtag = '#AI';
+-----+-----+-----+-----+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+-----+-----+-----+-----+
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
-> FROM Posts
-> WHERE hashtag = '#AI'
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-25';
+-----+-----+-----+-----+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+-----+-----+-----+-----+
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
-> FROM Posts
-> WHERE hashtag = '#AI'
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-25';
```

```

MySQL 8.0 Command Line Cli  x  +  v
+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
+-----+
1 row in set (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Posts
-> WHERE user_id = 101
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-31';
+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+
| 1 | SIMPLE | Posts | NULL | range | idx_user_timestamp | idx_user_timestamp | 13 | NULL | 3 | 100.00 | Using index condition |
+-----+
1 row in set, 1 warning (0.04 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Posts
-> WHERE hashtag = '#AI'
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-31';
+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+
| 1 | SIMPLE | Posts | NULL | range | idx_hashtag_timestamp | idx_hashtag_timestamp | 408 | NULL | 1 | 100.00 | Using index condition |
+-----+
1 row in set, 1 warning (0.04 sec)

mysql> SELECT *
-> FROM Posts
-> WHERE user_id = 101
-> AND hashtag = '#AI'
-> AND timestamp BETWEEN '2026-08-01' AND '2026-08-31';
+-----+
| post_id | user_id | post_text | timestamp | hashtag |
+-----+
| 2 | 101 | Learning AI | 2026-08-05 12:00:00 | #AI |
+-----+
1 row in set (0.00 sec)

```

1. Problem

A social media platform stores 500 million posts. The system needs fast queries based on:

- user_id — find posts created by a user
- timestamp — find recent posts or posts within a time range
- hashtag — find posts containing a particular hashtag

Typical queries are:

-- Posts by a user

SELECT *

FROM Posts

WHERE user_id = 101;

-- User's posts during a time period

SELECT *

FROM Posts

WHERE user_id = 101

AND timestamp BETWEEN '2026-01-01' AND '2026-01-31';

-- Posts containing a hashtag

```
SELECT *
```

```
FROM Posts
```

```
WHERE hashtag = '#technology';
```

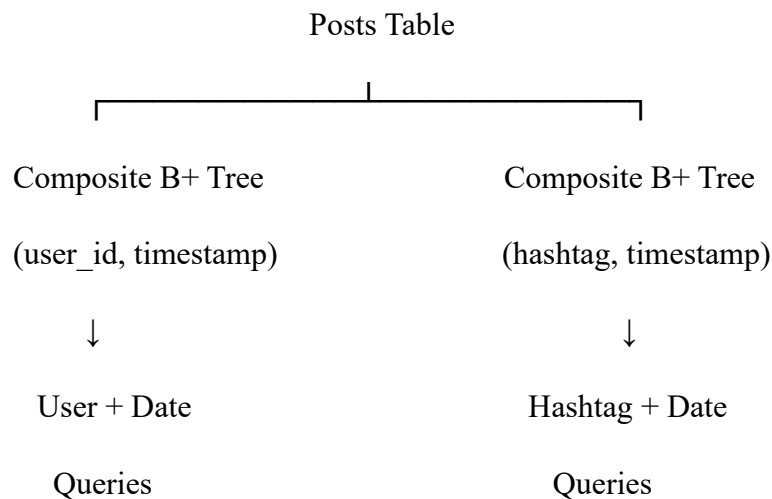
Because there are 500 million records, a full table scan is extremely expensive. Therefore, a carefully designed indexing strategy is required.

2. Proposed Composite Indexing Strategy

The main index should be:

Additionally, create a separate index for hashtags:

So the design is:



3. Why (user_id, timestamp)?

Consider the query:

```
SELECT *
```

```
FROM Posts
```

```
WHERE user_id = 101
```

```
AND timestamp BETWEEN '2026-08-01' AND '2026-08-25';
```

The index:

```
(user_id, timestamp)
```

first groups records by user_id.

Within each user, records are ordered by timestamp.

user_id

|

+-- 101

| |

| +-- Aug 01

| +-- Aug 05

| +-- Aug 10

| +-- Aug 20

|

+-- 102

|

+-- 103

Therefore, the database can quickly find user 101 and then scan only the required timestamp range.

4. Why (hashtag, timestamp)?

A social media platform frequently performs queries such as:

SELECT *

FROM Posts

WHERE hashtag = '#AI'

AND timestamp BETWEEN '2026-08-01' AND '2026-08-25';

The composite index:

(hashtag, timestamp)

allows the database to:

1. Find the hashtag.
2. Locate the relevant timestamp range.
3. Retrieve matching posts.

This is more efficient than scanning 500 million posts.

5. Why Not One Index (user_id, timestamp, hashtag)?

An index such as:

(user_id, timestamp, hashtag)

may look attractive, but it is not ideal for all three access patterns.

For example:

WHERE hashtag = '#AI'

cannot efficiently use the leading portion of an index whose first column is user_id.

This is called the leftmost-prefix rule.

Therefore, separate indexes are better for independent search patterns.

10) Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

ANS

```
MySQL 8.0 Command Line Cli
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SupermarketDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE SupermarketDB;
Database changed
mysql> CREATE TABLE HeapTransactions (
->   Transaction_ID BIGINT PRIMARY KEY,
->   Customer_ID INT,
->   Amount DECIMAL(10,2),
->   Transaction_Date DATETIME
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO HeapTransactions
-> VALUES
-> (100001, 501, 1250.50, '2026-08-25 10:30:00');
Query OK, 1 row affected (0.02 sec)

mysql> SELECT *
-> FROM HeapTransactions
-> WHERE Transaction_ID = 100001;
+-----+-----+-----+-----+
| Transaction_ID | Customer_ID | Amount | Transaction_Date |
+-----+-----+-----+-----+
| 100001 | 501 | 1250.50 | 2026-08-25 10:30:00 |
+-----+-----+-----+-----+
1 row in set (0.04 sec)
```

```

mysql> CREATE TABLE SortedTransactions (
  -> Transaction_ID BIGINT PRIMARY KEY,
  -> Customer_ID INT,
  -> Amount DECIMAL(10,2),
  -> Transaction_Date DATETIME
  -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO SortedTransactions
  -> VALUES
  -> (100001, 503, 900.00, '2026-08-25 11:00:00'),
  -> (100001, 501, 1250.50, '2026-08-25 10:30:00'),
  -> (100002, 502, 500.75, '2026-08-25 10:45:00');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM SortedTransactions
  -> ORDER BY Transaction_ID;
+-----+-----+-----+-----+
| Transaction_ID | Customer_ID | Amount | Transaction_Date |
+-----+-----+-----+-----+
| 100001 | 501 | 1250.50 | 2026-08-25 10:30:00 |
| 100001 | 503 | 900.00 | 2026-08-25 11:00:00 |
| 100002 | 502 | 500.75 | 2026-08-25 10:45:00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> CREATE TABLE HashedTransactions (
  -> Transaction_ID BIGINT PRIMARY KEY,
  -> Customer_ID INT,
  -> Amount DECIMAL(10,2),
  -> Transaction_Date DATETIME,
  -> Hash_Bucket INT
  -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.05 sec)

```

```

mysql> INSERT INTO HashedTransactions
  -> VALUES
  -> (
  -> 100001,
  -> 501,
  -> 1250.50,
  -> '2026-08-25 10:30:00',
  -> MOD(100001,1000)
  -> );
Query OK, 1 row affected (0.04 sec)

mysql> CREATE INDEX idx_hash_bucket
  -> ON HashedTransactions(Hash_Bucket);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM HashedTransactions
  -> WHERE Hash_Bucket = MOD(100001,1000)
  -> AND Transaction_ID = 100001;
+-----+-----+-----+-----+-----+
| Transaction_ID | Customer_ID | Amount | Transaction_Date | Hash_Bucket |
+-----+-----+-----+-----+-----+
| 100001 | 501 | 1250.50 | 2026-08-25 10:30:00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM HeapTransactions
  -> WHERE Transaction_ID = 100001;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | HeapTransactions | NULL | const | PRIMARY | PRIMARY | 8 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.04 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM SortedTransactions
  -> WHERE Transaction_ID = 100001;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | SortedTransactions | NULL | const | PRIMARY | PRIMARY | 8 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM HashedTransactions
  -> WHERE Hash_Bucket = MOD(100001,1000)
  -> AND Transaction_ID = 100001;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | HashedTransactions | NULL | const | PRIMARY, idx_hash_bucket | PRIMARY | 8 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

1. Problem

A supermarket billing system processes 1 million transactions per day.

Each transaction may contain:

- Transaction ID
- Customer ID
- Product ID
- Transaction date
- Amount

The system needs to perform three operations frequently:

1. Insert new transactions
2. Delete cancelled/old transactions
3. Search transactions by Transaction ID

We compare:

- Heap File
- Sorted File
- Hashed File

2. Heap File Organization

In a Heap File, records are stored in no particular order.

Block 1 → T101, T102, T103...

Block 2 → T104, T105, T106...

Block 3 → T107, T108, T109...

Insert

New records can simply be placed in available space.

Average complexity:

Therefore, Heap File is excellent for a billing system receiving 1 million new transactions every day.

Search

To find a transaction without an index, the system may need to scan the entire file.

For 1 million records, this can be expensive.

Delete

If the location of the record is known:

Otherwise, finding the record requires:

Summary

Operation	Heap
Insert	$O(1)$
Search	$O(N)$
Delete	$O(N)$ to locate + deletion

3. Sorted File Organization

Records are maintained in sorted order based on a key such as Transaction_ID.

1001

1002

1003

1004

1005

...

Insert

When a new transaction arrives, it must be placed at the correct position.

Records may need to be shifted.

Therefore:

in the worst case.

For 1 million daily transactions, this can become expensive.

Search

Since records are sorted, binary search can be used:

For:

approximately:

So only around 20 comparisons are needed theoretically for binary search.

Delete

Deleting a record while maintaining sorted order can require shifting records.

Summary

Operation	Sorted
Insert	$O(N)$
Search	$O(\log N)$
Delete	$O(N)$

4. Hashed File Organization

A hash function determines where a transaction is stored.

For example: where B is the number of buckets.

Transaction ID



Hash Function



Bucket



Transaction

Example:

Transaction ID = 105025

$105025 \text{ MOD } 1000 = 25$

Transaction $\rightarrow$ Bucket 25

Insert

The hash function directly determines the bucket.

Average:

Search

The same hash function finds the bucket.

Average:

Delete

The bucket can be located using the hash function.

Average:

However, collisions can increase the cost.

5. Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$	$O(N)$	$O(1)$ average
Search	$O(N)$	$O(\log N)$	$O(1)$ average
Delete	$O(N)$	$O(N)$	$O(1)$ average
Best use	Heavy insertion	Ordered/range queries	Exact-match queries

6. Best Choice for Supermarket Billing

A supermarket generates:

transactions per day.

The workload is therefore write-heavy.

If the most common query is:

Find transaction using Transaction_ID

then Hashing is the best choice.

Why?

- Very fast insertion
- Very fast exact search
- Very fast deletion
- Average $O(1)$ performance
- Suitable for large transaction volumes